

Assignment 6 — Solution Key

Hardwired Control Unit Design

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

August 8, 2026

6.1 Conceptual

Q1. D_i alone identifies *which instruction* is running, but says nothing about *when* in that instruction's execution we are; T_j alone identifies *which step* is active, but says nothing about *which instruction* is running. A signal like Y_{in} must fire at T_1 for ADD but must *not* fire at T_1 for, say, LOAD, which needs something different at its own T_1 . Only the conjunction $D_{ADD} \cdot T_1$ correctly captures “this is ADD, and we are at its first step” — neither factor alone can express that.

Q2. Without SC_{clear} , SC would simply keep incrementing forever after an instruction's last step ($T_4, T_5, \dots$), since nothing resets it back to 0. The step decoder would dutifully produce one-hot lines for these later steps, but no instruction's state table defines any control signals for T_4 or beyond, so the datapath would simply idle: no signal would fire, and the next instruction would never be fetched. The CPU would therefore execute exactly **one instruction** and then freeze — SC_{clear} is what allows the control unit to cycle through a program at all.

6.2 Numerical

Q3. 12 instructions need **12 one-hot D_i lines** (one output line per instruction, by definition of one-hot encoding). The opcode field needs $\lceil \log_2 12 \rceil = 4$ **bits** (4 bits address up to 16 distinct codes, which is the smallest power of two ≥ 12 ; 3 bits would only address 8, which is insufficient).

Q4. A 5-bit SC can hold $2^5 = 32$ distinct binary values, so it can address up to **32 distinct control steps** (T_1 through T_{32} in this course's 1-indexed convention).

6.3 Design

Q5. SUB has the identical three-step shape as ADD, differing only in the ALU function selected at T_2 :

Step	RTL	Control signals active
T_1	$Y \leftarrow [R5]$	$R5_{out}, Y_{in}$
T_2	$Z \leftarrow [Y] - [R6]$	$R6_{out}, ALU=subtract, Z_{in}$
T_3	$R4 \leftarrow [Z]$	$Z_{out}, R4_{in}, SC_{clear}$

Boolean equations (opcode line D_{SUB}):

$$Y_{in} = D_{SUB} \cdot T_1, \quad Z_{in} = D_{SUB} \cdot T_2, \quad R4_{in} = D_{SUB} \cdot T_3, \quad SC_{clear} = D_{SUB} \cdot T_3.$$

(If **ADD** also asserts Z_{in} at its own T_2 , as in the lecture's worked example, the full equation for Z_{in} across both instructions is $Z_{in} = (D_{ADD} \cdot T_2) + (D_{SUB} \cdot T_2)$, following the same OR-across-instructions pattern.)

Q6. State table for **LOAD R1, (R2)**:

Step	RTL	Control signals active
T_1	$MAR \leftarrow [R2]$	$R2_{out}, MAR_{in}$
T_2	$R1 \leftarrow M[MAR]$	MemRead, $R1_{in}, SC_{clear}$

MAR_{in} fires only at $D_{LOAD} \cdot T_1$, so if no other instruction's state table lists MAR_{in} at T_1 , the equation is simply $MAR_{in} = D_{LOAD} \cdot T_1$, realized as a **single AND gate** with inputs D_{LOAD} and T_1 and output MAR_{in} — no OR gate is needed unless a second instruction shares this exact signal at this exact step, in which case a second AND gate (for that instruction's $D_i \cdot T_1$ term) would feed the same OR gate, following the pattern established for Z_{in} in the lecture.